
PORTFOLIO - GRADUAAT PROGRAMMEREN

Persoonlijk Dashboard

Een webapplicatie met vier API-gedreven widgets

De programmeur · Doel 2: Je programmeert volgens de standaarden

Auteur Jonas Trap

Opleiding Graduaat Programmeren, Thomas More Geel

Academiejaar 2025-2026

Type bewijs eigen project

Het project

Dit dashboard is een persoonlijk project: een single-page webapplicatie die vier dagelijkse noden samenbrengt op één centrale pagina. De vier widgets tonen het actuele weerbericht voor opgeslagen locaties, een dagelijks avondmaalrecept met filters voor dieet en bereidingstijd, de NASA-foto van de dag met uitleg, en een eenvoudig takenbeheer.

Het project dient als demonstratiestuk van mijn vaardigheden in webontwikkeling en API-integratie. De nadruk lag bewust op een nette, professionele uitwerking die de geleerde concepten concreet toepast: een duidelijke architectuur, type-veiligheid, integratie met externe API's en degelijke foutafhandeling.

Technische keuzes

- **Build tool:** Vite, voor een snelle ontwikkelomgeving met native TypeScript-ondersteuning.
- **Taal:** TypeScript, voor type-veiligheid en betere onderhoudbaarheid.
- **CSS-framework:** Bootstrap 5, voor een degelijke grid en ingebouwde dark mode.
- **API's:** OpenWeatherMap (weer), Spoonacular (recepten), NASA APOD (foto van de dag), en een vertaal-API voor de Nederlandse inhoud.

Broncode

De volledige broncode van dit project staat publiek op GitHub, zodat ze altijd terug te vinden is: github.com/jonastapmore/jonastap-projecten/tree/main/dashboard

Werken volgens de standaarden

Ik koppel dit project aan het doel "*je programmeert volgens de standaarden*" omdat ik doorheen het hele project consequent professionele conventies en gangbare patronen heb toegepast, in elke laag van de applicatie: TypeScript, HTML, CSS, API-integratie en foutafhandeling.

TypeScript

- Strikte modus aan (`strict: true` in `tsconfig.json`), zodat fouten al tijdens het ontwikkelen opduiken.
- Voor elk API-antwoord een eigen **interface** (bv. `WeatherResponse`, `Recipe`, `ApodResponse`), zodat de data getypeerd en voorspelbaar is.
- **Private velden** (#) en kleine methodes met één duidelijke verantwoordelijkheid.
- Modern `async/await` in plaats van losse callbacks.

Betekenisvolle namen (variabelen, objecten, functies en methoden)

- **Klassen en objecten** beschrijven hun rol: `WeatherWidget`, `RecipeWidget`, het model `SavedLocation`, en de provider `locationsLocalStorageProvider`.
- **Functies en methoden** zeggen wat ze doen: `#getRandomRecipe()`, `#showMeal()`, `#renderIngredients()`, `#showError()`, `#weatherEmoji()` en de gedeelde `translate()`.
- **Variabelen** zijn sprekend: `vertaaldeRegels`, `gekozen`, `calorieën`, `perDag`, in plaats van nietszeggende namen als `x` of `data1`.

HTML

- Semantische opbouw en herbruikbare componenten via **custom elements** (elke widget is een eigen web component).
- Toegankelijkheid: `aria-label`'s op knoppen en invoervelden, betekenisvolle alt-teksten op afbeeldingen.

CSS

- **CSS-variabelen** voor de huisstijl (accentkleur, achtergrond), zodat het thema centraal beheerd wordt.
- Een **licht/donker-thema** dat via een toggle wisselt, met Bootstrap's `data-bs-theme`.
- **Responsive** ontwerp met de Bootstrap-grid en eigen media queries.

API-integratie

- Aanvragen met de browser `fetch`-API, met `URLSearchParams` voor nette querystrings.
- Getypeerde verwerking van de JSON-antwoorden.

- Een **gedeelde vertaalhulp** die door meerdere widgets hergebruikt wordt (DRY), met een fallback van Google naar MyMemory.
- API-sleutels in een `.env`-bestand, nooit hardgecodeerd.

Foutafhandeling

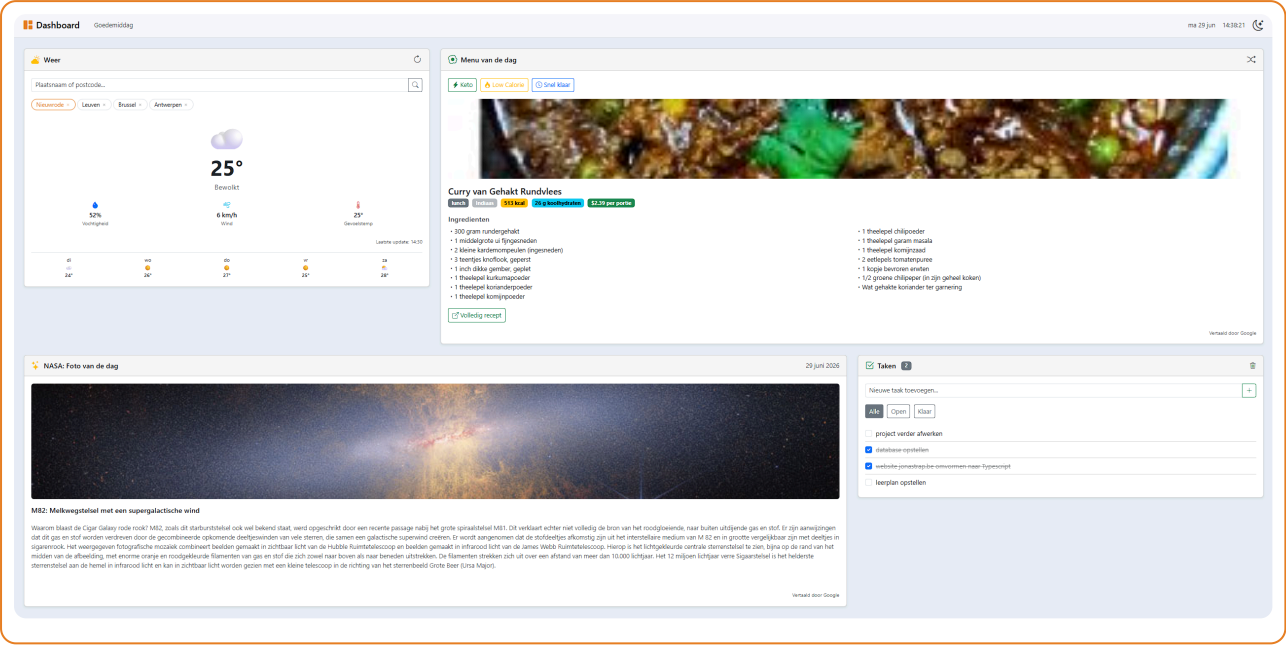
- try/catch rond elke API-aanvraag, met **gebruiksvriendelijke meldingen** in plaats van een blanco scherm.
- **Automatische herhaling** bij een tijdelijke serverfout (bv. NASA die af en toe een 503 geeft).
- Specifieke meldingen per situatie: dagelijkse limiet bereikt, geen resultaat voor de filters, of een netwerkfout.

Overzichtelijke structuur, onderhoud en hergebruik

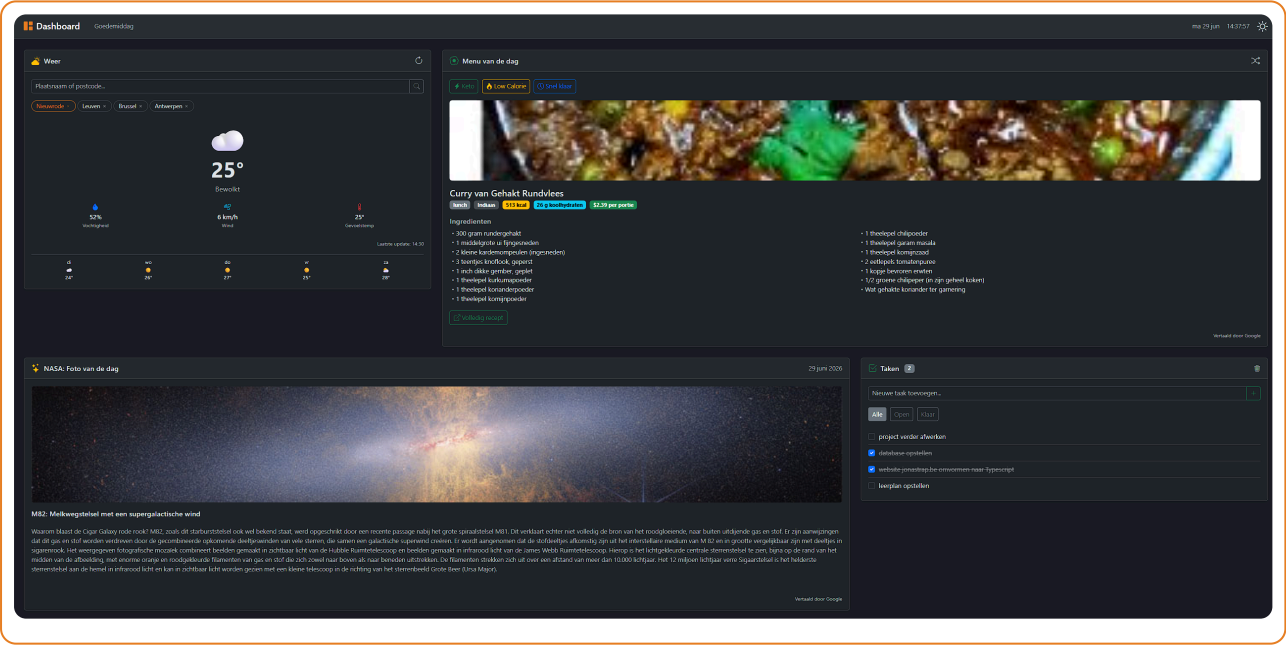
Ik werk overzichtelijk en gestructureerd, zodat onderhoud en hergebruik vanzelfsprekend zijn:

- Een duidelijke laagopbouw (components, data, effects, models, pages, router), zodat je meteen weet waar iets thuishoort.
- Een abstracte PersistenceProvider met twee implementaties (localStorage en memory) en het **observer-patroon** voor automatisch herrenderen.
- Versiebeheer met **git** en een `.gitignore` die `node_modules`, `.env` en `dist` buiten de repository houdt.

Het resultaat



Afbeelding 1: het dashboard in het lichte thema, met de vier widgets (weer, recept, NASA, taken).



Afbeelding 2: hetzelfde dashboard in het donkere thema. De theme-toggle in de navbar wisselt centraal tussen licht en donker, wat de CSS-standaard (themabeheer) aantoont.

Mapstructuur

De duidelijke laagopbouw maakt het project overzichtelijk en uitbreidbaar:

```
dashboard/  
  src/  
    components/      widgets en navbar (custom elements)  
      weatherWidget/  
      recipeWidget/  
      nasaWidget/  
      notesWidget/  
      navbar/  
    data/             persistence providers (abstract, localStorage, memory)  
    effects/          thema-beheer en de gedeelde vertaalhulp  
    models/           datamodellen (Task, SavedLocation)  
    pages/            de dashboardpagina  
    router/           Router, Page en CustomElement basisklassen  
    main.ts           registratie van de widgets en de routes  
    style.css  
  index.html  
  package.json  
  tsconfig.json  
  vite.config.ts
```

Voorbeeld van code volgens de standaarden

De gedeelde vertaalhulp toont meerdere standaarden tegelijk: een getypeerd resultaat, `async/await`, foutafhandeling met een fallback, en duidelijke comments.

```
// Resultaat van een vertaling: de tekst plus welke engine ze leverde  
export interface VertaalResultaat {  
  text: string;  
  engine: string;  
}  
  
// Probeert eerst Google, valt terug op MyMemory, dan op de originele tekst  
export async function translate(text: string, from = 'en', to = 'nl'):  
  Promise<VertaalResultaat> {  
  if (!text || text.trim() === '') {  
    return { text, engine: 'Origineel (Engels)' };  
  }  
  try {  
    const res = await  
      fetch(`https://translate.googleapis.com/...&q=${encodeURIComponent(text)}`);  
    if (res.ok) {  
      const data = await res.json();  
      const vertaald = data[0].map((segment) => segment[0]).join('');  
      return { text: vertaald, engine: 'Google' };  
    }  
  } catch {  
    // val door naar MyMemory  
  }  
}
```

```
// ... MyMemory-fallback ...  
return { text, engine: 'Origineel (Engels)' };  
}
```

Codefragment uit `src/effects/translate.ts`: getypeerd, sprekende namen (`VertaalResultaat`, `vertaald`, `segment`) en foutafhandeling met `fallback`.

Betekenisvolle namen en een herbruikbare structuur

Elke widget volgt dezelfde, overzichtelijke opbouw met sprekende methodenamen, wat onderhoud en uitbreiding eenvoudig maakt:

```
export class RecipeWidget extends CustomElement {  
  constructor() {  
    super(HTML);  
    // luisteraars koppelen en meteen een recept laden  
  }  
  
  async #getRandomRecipe() { /* haalt een recept op, rekening houdend met de filters */ }  
  async #showMeal(recipe: Recipe) { /* vult de kaart met het recept */ }  
  async #renderIngredients(recipe: Recipe) { /* bouwt de ingredientenlijst op */ }  
  #showError(bericht: string) { /* toont een nette foutmelding */ }  
}
```

Vereenvoudigd fragment uit `src/components/recipeWidget/recipeWidget.ts`: de methodenamen verraden meteen hun functie, en elke widget volgt dezelfde structuur.

Zelfreflectie

Wat ging goed

Door vanaf het begin één vast patroon aan te houden (elke widget als custom element met dezelfde opbouw), kon ik elke nieuwe widget snel en herkenbaar uitwerken. De strikte TypeScript-modus ving regelmatig typfouten op nog voor ik de app draaide, bijvoorbeeld verkeerd gespelde veldnamen in API-antwoorden. Het hergebruik van de provider en de vertaalhulp hield de code DRY.

Wat ik geleerd heb

Ik heb gemerkt hoe sterk standaarden en een vaste structuur het werk vergemakkelijken: uitbreiden werd voorspelbaar omdat alles hetzelfde stramien volgde. Bij de API-integraties leerde ik het belang van getypeerde antwoorden en degelijke foutafhandeling: externe diensten zijn niet altijd beschikbaar (NASA gaf soms een 503, Spoonacular een limietfout), en een nette melding of een automatische herhaling maakt het verschil tussen een professionele en een kapotte indruk.

Uitdagingen

Een aandachtspunt was de beveiliging van API-sleutels. Ze staan keurig in een `.env`-bestand en buiten versiebeheer, maar in een frontend-applicatie belanden ze bij het bouwen toch in de browsercode. Ik begrijp nu dat een echte oplossing een backend-proxy vereist die de sleutel verbergt. Voor een lokaal demonstratieproject is de huidige aanpak aanvaardbaar, maar ik weet wat de professionele standaard zou zijn.

Wat beter kan

- Een **linter** (ESLint/Prettier) toevoegen om de standaarden automatisch af te dwingen, in plaats van ze handmatig te bewaken.
- **Automatische tests** schrijven; nu heb ik enkel handmatig getest.
- De sleutels achter een **backend-proxy** plaatsen voor een echte productie-omgeving.

Conclusie

Dit project toont aan dat ik volgens de standaarden programmeer: getypeerde, gestructureerde code met herkenbare patronen, toegankelijke HTML, beheerde CSS, nette API-integratie en degelijke foutafhandeling, allemaal onder versiebeheer. De punten die beter kunnen, ken ik en kan ik benoemen, wat aantoont dat ik de standaard niet alleen toepas maar ook begrijp.